Understanding PID Control from Scratch

Intuition, Mathematics, Python Simulations, and Real-World Engineering Fixes

What This Guide Covers: PID (Proportional–Integral–Derivative) control is the workhorse algorithm behind cruise control, drones, robot arms, 3D printers, and industrial motors. This guide teaches PID from zero prior background: every topic begins with a plain-English idea, backs it up with exact formulas, and proves it with reproducible Python (NumPy + Matplotlib) graphs.

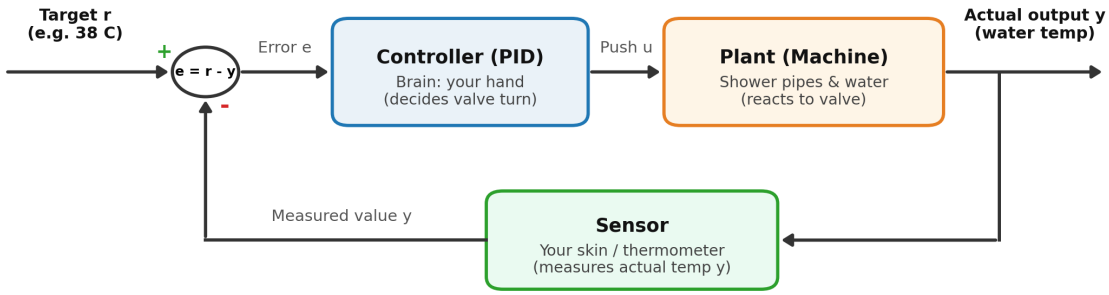
1. The Basic Idea

Imagine adjusting a shower to a comfortable **38 °C** when the water starts freezing cold at 15 °C. You do not calculate pipe fluid dynamics in your head. Instead, you repeat a simple three-step loop:

1. **Measure:** Feel the water temperature with your hand (*sensor*).
2. **Compare:** Notice how far the actual temperature is from your 38 °C goal (*error* $= 38 - 15 = +23$ °C).
3. **Act:** Turn the hot valve (*controller*) by a lot when it is far off, a little when it is close, and back off if it warms up too fast.

In engineering, the machine being controlled (pipes, motor, car, or spring-mass) is called the **plant**. The desired goal is the **setpoint** $r(t)$, the measured output is $y(t)$, and the gap $e(t) = r(t) - y(t)$ is the **error**. Because the sensor feeds the output back to the input to compute the error, this is called a **closed-loop feedback system** (Figure 1).

Figure 1: Closed-Loop Feedback Control Diagram (with Shower Temperature Analogy)



Takeaway: A feedback controller continuously measures the output y, compares it to target r, and adjusts effort u until error e = 0.

Figure 1: Closed-loop feedback structure: the controller continuously shrinks the gap $e = r - y$ using live sensor feedback.

2. The PID Formula

A PID controller turns the error $e(t) = r(t) - y(t)$ into a control command $u(t)$ by adding three simple reactions—to the **present**, the **past**, and the **trend (future)** of the error:

$$u(t) = \underbrace{K_p\, e(t)}_{\text{P: present error}} + \underbrace{K_i \int_0^t e(\tau)\, d\tau}_{\text{I: accumulated past error}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{D: rate of change}} \tag{1}$$

What each letter means and its units (illustrated for our position test system, where position y is in meters m, time t is in seconds s, and motor push u is in Newtons N; for other plants, units scale with output and actuator units):

- $r(t)$ (m): **Setpoint** (target position). $y(t)$ (m): **Measured output** (actual position).
- $e(t) = r(t) - y(t)$ (m): **Error** (positive means we are short of the target; negative means we overshot).
- $u(t)$ (N): **Control effort** sent to the actuator/motor.
- K_p (N/m): **Proportional gain**. Pushes harder the larger the *present* error is (like a virtual spring pulling toward the target).
- K_i (N/(m · s)): **Integral gain**. Adds up all *past* error over time; even a tiny lingering gap eventually builds enough push to erase steady-state error.
- K_d (N · s/m): **Derivative gain**. Reacts to *how fast the error is changing*; as the machine races toward the target, $\frac{de}{dt}$ is negative, so D applies a braking force to prevent overshoot.

How a computer runs PID step by step: A microcontroller wakes up every small time step Δt (for example, $\Delta t = 0.005$ s at 200 Hz). Replacing the integral with a running sum and the derivative with a slope between consecutive samples gives the discrete algorithm at step k:

$$e_k = r_k - y_k, \qquad I_k = I_{k-1} + e_k\,\Delta t, \qquad D_k = \frac{e_k - e_{k-1}}{\Delta t}, \qquad u_k = K_p e_k + K_i I_k + K_d D_k. \tag{2}$$

3. The Test System (Given)

To test every PID concept with real numbers, we simulate a **1 kg block on a spring with a damper** (just like one corner of a car suspension): mass $m = 1$ kg, viscous damping $c = 10$ N · s/m, and spring stiffness $k = 20$ N/m. Starting at rest ($x(0) = 0$ m, $\dot{x}(0) = 0$ m/s), the goal is to move and hold the block at $r = 1.0$ m.

By Newton's second law ($m\ddot{x} = \sum F$), the motor force $u(t)$ pushes forward while the damper resists motion ($-c\dot{x}$) and the spring pulls back ($-kx$):

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = u(t) \quad \Longrightarrow \quad \ddot{x}(t) + 10\dot{x}(t) + 20x(t) = u(t). \tag{3}$$

For numerical integration (4th-order Runge–Kutta in `systems.py`), we rewrite Eq. (3) as two first-order equations for position x and velocity $v = \dot{x}$:

$$\dot{x} = v, \qquad \dot{v} = \frac{u - 10v - 20x}{1}.$$

Why the spring needs a constant push to stay at 1 m: Once the block reaches $x = 1.0$ m and stops moving ($\dot{x} = 0$, $\ddot{x} = 0$), the damper force vanishes ($10 \times 0 = 0$), but the stretched spring pulls backward with $kx = 20 \times 1.0 = 20$ N. Setting $\dot{x} = \ddot{x} = 0$ in Eq. (3) shows that **the motor must keep pushing with a constant $u_\infty = 20$ N forever**, otherwise the spring yanks the block back toward 0 m. Figure 2 confirms this open-loop test: 20 N holds at 1.00 m, whereas 10 N stops halfway at 0.50 m.

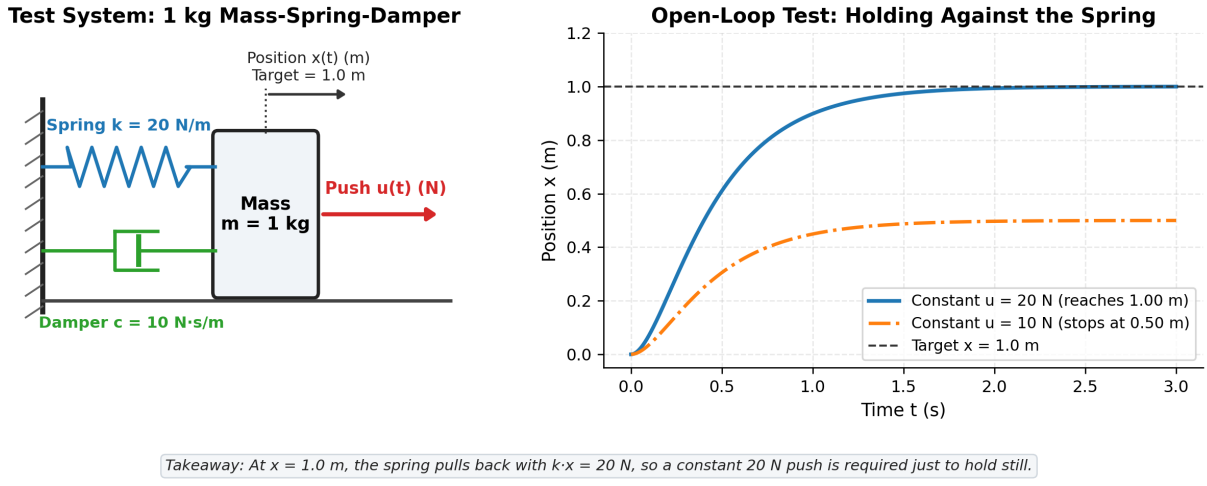


Figure 2: Left: 1 kg mass-spring-damper test plant. Right: open-loop step tests proving $u = 20$ N is required to hold $x = 1.00$ m.

4. How to Judge a Result

Before tuning gains, we need four clear numbers to judge any step response (moving from 0 m toward target $r = 1.0$ m):

1. **Rise time (t_r, s):** How fast it rises—the time from reaching 10% of its final steady value y_{ss} to reaching 90% of y_{ss}.
2. **Overshoot (%OS, %):** How far past y_{ss} it swings before turning back: $\%\text{OS} = \max\left(0, \frac{x_{peak} - y_{ss}}{y_{ss}}\right) \times 100\%$.
3. **Settling time (t_s, s):** How long until bouncing dies out and $x(t)$ stays inside the $\pm 2\%$ band around y_{ss} forever.
4. **Steady-state error (e_{ss}, m):** How far from the target r it finally stops: $e_{ss} = |r - y_{ss}|$.

Figure 3 marks all four measures on a single step response of our test system using P-only control ($K_p = 180$): it rises fast ($t_r = 0.098$ s), overshoots by 30.9% (peaking at 1.178 m), settles in $t_s = 0.777$ s, and stops short at $y_{ss} = 0.90$ m ($e_{ss} = 0.10$ m).

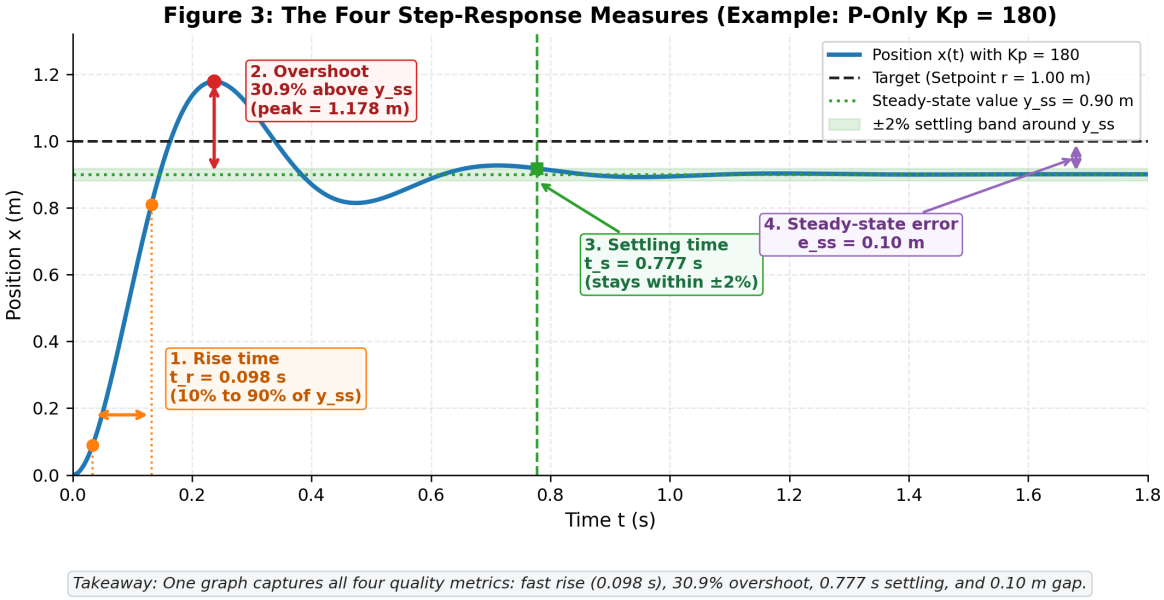


Figure 3: All four step-response measures annotated on a single response ($K_p = 180$).

5. Each Part Alone (With Analytical Proofs)

5.1 Proportional Only (P): Fast, Simple, Always Stops Short

With $K_i = K_d = 0$, the controller applies $u(t) = K_p(r - x)$. Plugging this into Eq. (3) gives:

$$\ddot{x} + 10\dot{x} + (20 + K_p)x = K_p r. \tag{4}$$

Exact steady-state calculation: At steady state, motion stops ($\dot{x} = \ddot{x} = 0$), so $(20 + K_p)x_{ss} = K_p r$:

$$x_{ss} = \frac{K_p}{K_p + k} r = \frac{K_p}{K_p + 20} \text{ m}, \qquad e_{ss} = r - x_{ss} = \frac{20}{K_p + 20} \text{ m}. \tag{5}$$

Why can P alone *never* reach 1.0 m? Because at $x = 1.0$ m the error is $e = 0$, so a P-only controller pushes with $u = K_p \cdot 0 = 0$ N—yet Section 3 proved the spring needs 20 N! So the block must settle below 1 m where $K_p e_{ss} = 20\, x_{ss}$. Table 1 and Figure 4 show exact agreement between Eq. (5) and our simulation across four strengths ($K_p \in \{10, 40, 100, 200\}$): raising K_p shrinks e_{ss} from 0.667 m to 0.091 m and speeds up rise time (0.537 s $\rightarrow$ 0.092 s), but triggers 32.9% overshoot at $K_p = 200$.

K_p (N/m)	Formula x_{ss} (m)	Simulated x_{ss} (m)	e_{ss} (m)	Rise time t_r (s)	Overshoot (%)	Settling t_s (s)
10	0.3333	0.3333	0.6667	0.537	0.09	0.883
40	0.6667	0.6667	0.3333	0.254	7.09	0.776
100	0.8333	0.8333	0.1667	0.142	20.16	0.761
200	0.9091	0.9091	0.0909	0.092	32.88	0.749

Table 1: P-only analytical vs. simulated results: steady-state predictions match simulation to four decimal places.

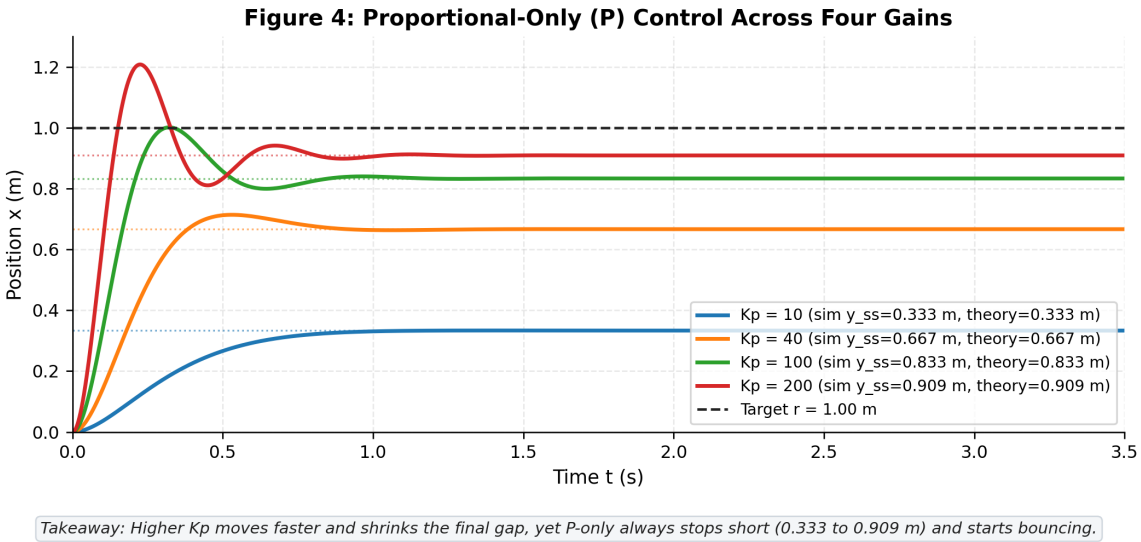


Figure 4: P-only responses for $K_p \in \{10, 40, 100, 200\}$ with theoretical steady-state dotted lines.

5.2 Adding Integral (PI): Erasing the Final Gap

To supply the constant 20 N holding force even when $e = 0$, we add the I term ($K_p = 60$, varying $K_i \in \{0, 60, 150, 350\}$). As long as $x < 1$ m, the running integral $\int e\,dt$ keeps growing, pushing the block up until $e = 0$ and the stored integral holds $K_i \int_0^\infty e(\tau)\,d\tau = 20$ N. As Figure 5 shows, adding $K_i = 150$ removes the entire 0.250 m steady-state gap ($e_{ss} = 0.0000$ m), though pushing $K_i = 350$ makes the controller wind up too much force early on and overshoot by 46.8%.

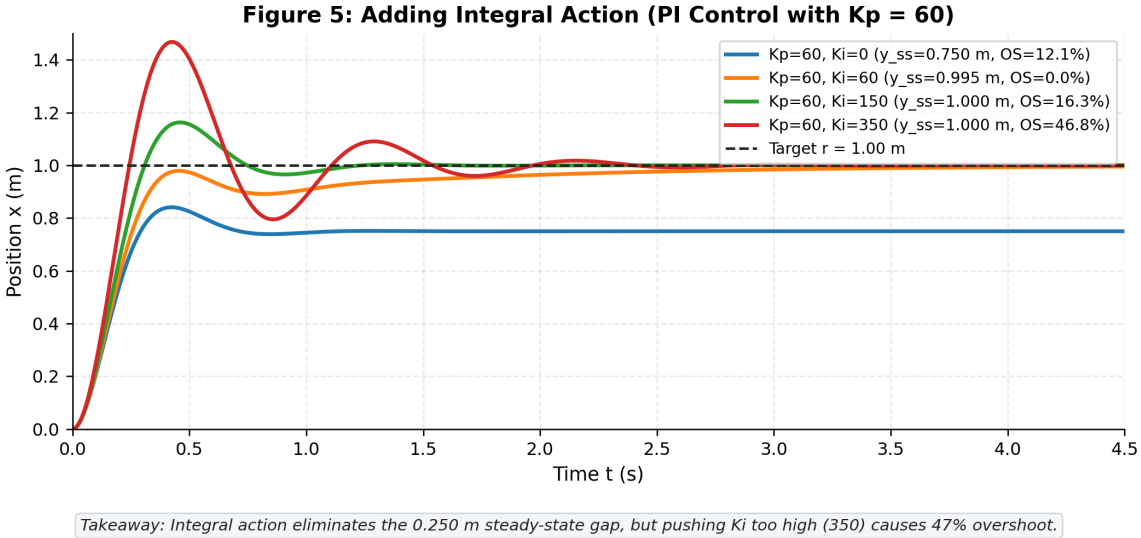


Figure 5: PI control ($K_p = 60$): K_i closes the 0.250 m gap completely, while excessive $K_i = 350$ causes 46.8% overshoot.

5.3 Adding Derivative (PD): Braking the Bounce

With $u = K_p(r - x) + K_d \frac{d}{dt}(r - x)$ and a constant target ($\dot{r} = 0$), $\dot{e} = -\dot{x}$, so Eq. (3) becomes:

$$\ddot{x} + \underbrace{(10 + K_d)}_{c_{\text{eff}}}\dot{x} + (20 + K_p)x = K_p r. \tag{6}$$

Exact damping calculation: Comparing Eq. (6) with standard second-order form $\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2 x = \ldots$, the derivative gain adds directly to physical damping ($c_{\text{eff}} = 10 + K_d$). For $K_p = 200$, $\omega_n = \sqrt{20 + 200} = \sqrt{220} = 14.832$ rad/s, so:

$$\zeta = \frac{10 + K_d}{2\sqrt{220}}, \qquad \%\text{OS}_{\text{theory}} = \exp\left(-\frac{\pi\zeta}{\sqrt{1 - \zeta^2}}\right) \times 100\% \quad (\zeta < 1). \tag{7}$$

Table 2 and Figure 6 verify Eq. (7): increasing K_d from 0 to 15 raises ζ from 0.337 to 0.843, slashing overshoot from 32.88% to 0.69% and cutting settling time from 0.749 s to 0.278 s (the tiny 0.4% difference at $K_d = 0$ comes from the 1 ms zero-order hold step).

K_d (N · s/m)	Effective c_{eff}	Damping ratio ζ	Theory %OS	Simulated %OS	Settling t_s (s)
0	10	0.3371	32.47%	32.88%	0.749
5	15	0.5056	15.86%	16.03%	0.541
15	25	0.8427	0.73%	0.69%	0.278
25	35	1.1798	0.00%	0.00%	0.522

Table 2: PD control ($K_p = 200$): derivative action increases ζ and suppresses bouncing exactly as Eq. (7) predicts.

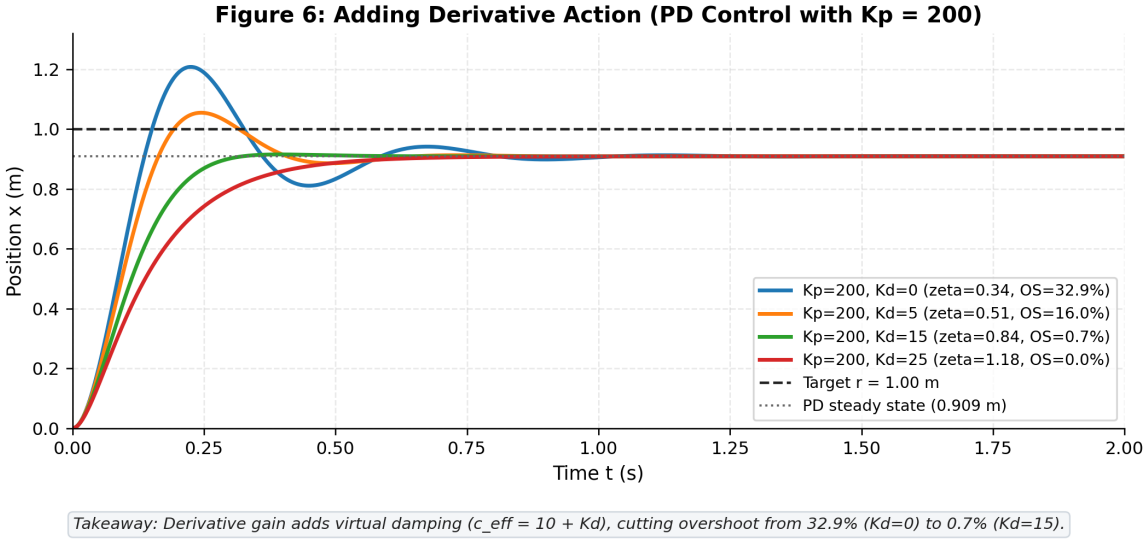


Figure 6: PD control ($K_p = 200$): raising K_d damps oscillations from 32.9% overshoot down to 0.0%.

6. All Together: P vs. PI vs. PD vs. Full PID

Keeping $K_p = 100$ fixed so the comparison is fair, Figure 7 and Table 3 compare P, PI, PD, and full PID on the same $0 \to 1$ m task:

- **P** $(100, 0, 0)$ rises fast (0.142 s) but bounces (20.16% OS) and stops 0.167 m short.
- **PI** $(100, 120, 0)$ eliminates the gap (0.007 m at 2.5 s, 0 m asymptotically) but oscillates (14.57% OS, slow 1.325 s settling).
- **PD** $(100, 0, 8)$ tames overshoot to 1.06% and settles in 0.359 s, yet still stops 0.167 m short.
- **PID** $(100, 120, 8)$ combines every strength: fast rise (0.261 s), under 1% overshoot (0.86%), rapid settling (0.408 s), and virtually zero error (0.0001 m).

Final gain choice: For the unconstrained test system, we choose $K_p = 100$ **N/m**, $K_i = 120$ **N/(m · s)**, $K_d = 8$ **N · s/m** because it achieves sub-1% overshoot (0.86%) and settles in 0.408 s without excessive peak force (100.1 N). (When the motor is capped at ±30 N in Sections 7 and 9, we also test a more aggressive tuning $(120, 240, 12)$ to demonstrate how anti-windup rescues strong integral gains.)

Controller (K_p, K_i, K_d)	Rise time t_r (s)	Overshoot (%)	Settling t_s (s)	Steady error e_{ss} (m)	Peak force (N)
P $(100, 0, 0)$	0.142	20.16%	0.761	0.1667	100.0
PI $(100, 120, 0)$	0.157	14.57%	1.325	0.0067	100.9
PD $(100, 0, 8)$	0.232	1.06%	0.359	0.1667	100.0
PID $(100, 120, 8)$	**0.261**	**0.86%**	**0.408**	**0.0001**	**100.1**

Table 3: Step-response comparison of P, PI, PD, and full PID on the 1 kg mass-spring-damper ($t_{\text{end}} = 2.5$ s).

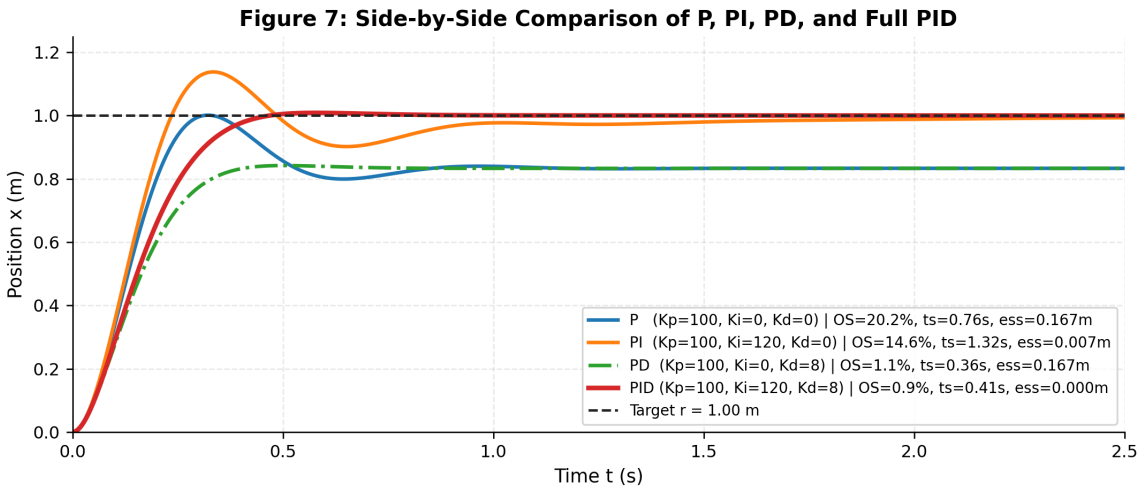


Figure 7: P, PI, PD, and full PID compared directly: only PID hits 1.00 m fast without bouncing.

7. Real-World Problems (and How to Fix Them)

Ideal math ignores motor limits, noisy sensors, instant target jumps, and finite computer speed. Here are the four classic pitfalls on our test system (using PID $(120, 240, 12)$) and the exact engineering fix for each.

(a) Actuator Limit (± 30 N) and Integrator Windup

Problem: Real motors saturate. If the motor can push at most 30 N, the block takes longer to reach 1 m. Meanwhile, a naive integrator keeps summing error and "winds up" to 68.6 N—more than double the motor limit. Even after the block passes 1.0 m, the huge stored I term keeps pinning the motor at +30 N, causing **26.9% overshoot** and 2.05 s settling.

Fix (Conditional Integration / Anti-Windup): Stop adding to the integral whenever the output is already saturated and the error pushes further into saturation. Result (Figure 8): overshoot drops from **26.9% to 0.0%** and settling time improves from **2.05 s to 0.83 s**.

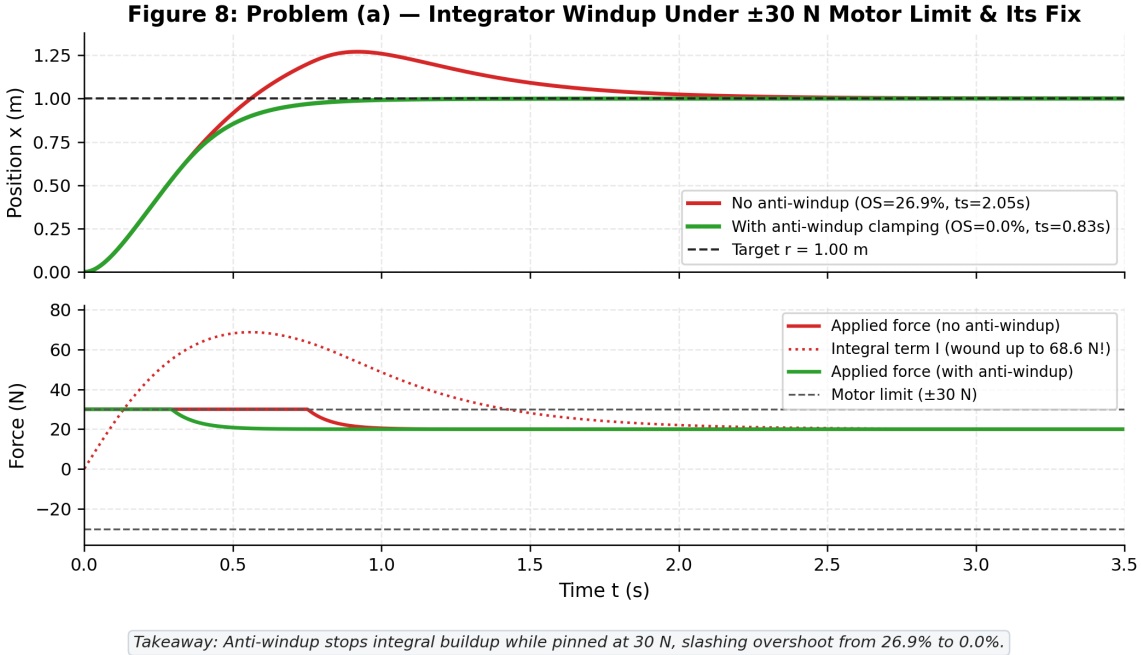


Figure 8: Integrator windup under a ± 30 N limit vs. conditional-integration anti-windup.

(b) Noisy Sensor (±1 mm) and Jumpy Derivative

Problem: Suppose an encoder has ±1 mm (0.001 m) measurement noise at 1 kHz ($\Delta t = 0.001$ s). Dividing a 1 mm jump by 0.001 s produces ±1 m/s fake velocity spikes, which $K_d = 12$ amplifies into **39.7 N peak-to-peak** motor chatter ($\sigma = 6.86$ N) even while sitting still!

Fix (Low-Pass Derivative Filter): Smooth the derivative using a first-order low-pass filter with time constant $\tau_d = 0.025$ s: $D_k = \frac{\tau_d D_{k-1} + \Delta t\, d_k}{\tau_d + \Delta t}$. Result (Figure 9): steady-state motor chatter drops by **96.8%**, from 39.7 N down to **1.26 N** peak-to-peak ($\sigma = 0.24$ N).

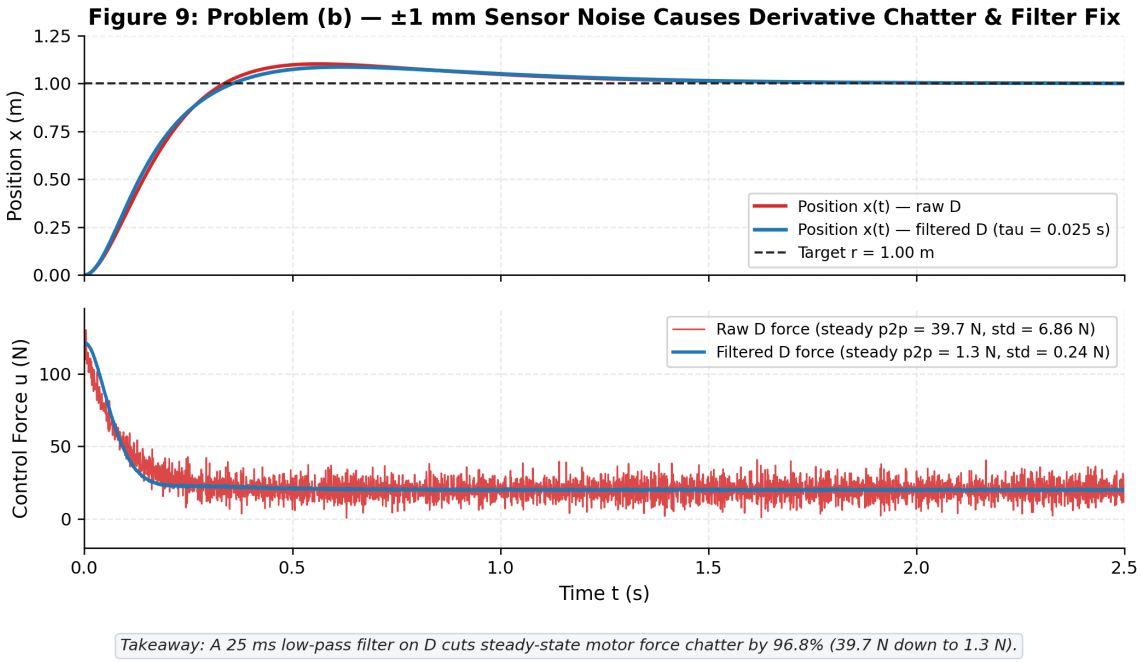


Figure 9: Raw derivative turns ±1 mm sensor noise into 39.7 N force chatter; a 25 ms filter cuts it by 96.8%.

(c) Sudden Target Step and Derivative Kick

Problem: When the setpoint jumps instantly from 0 to 1 m at $t = 0.5$ s (with $\Delta t = 0.001$ s), $\frac{e_k - e_{k-1}}{\Delta t}$ becomes $\frac{1.0}{0.001} = 1000$ m/s, firing a massive **12,120 N spike** (over 100× normal force) that can trip breakers or strip gears.

Fix (Derivative on Measurement): Since $\frac{de}{dt} = \frac{dr}{dt} - \frac{dy}{dt}$ and the physical block cannot teleport, compute D purely from $-\frac{y_k - y_{k-1}}{\Delta t}$. Result (Figure 10): the 12,120 N spike vanishes completely, capping peak force at **120.2 N**.

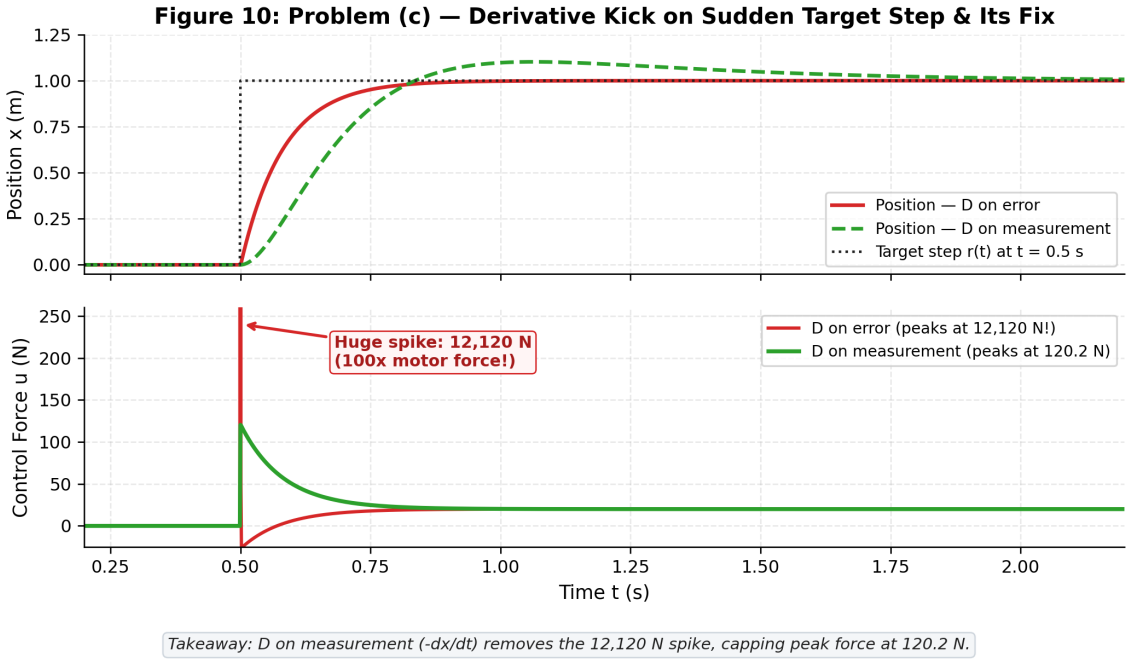


Figure 10: Derivative-on-measurement eliminates the 12,120 N setpoint kick.

(d) Slow Computer Update Rate (Δt Too Large)

Problem: Holding an outdated force command between slow updates injects an average delay of $\Delta t/2$, eroding phase margin. On our system (rise time ≈ 0.23 s), Figure 11 shows that $\Delta t = 0.001$ s (1000 Hz) and $\Delta t = 0.060$ s (17 Hz) behave smoothly (10.2% and 8.4% overshoot), $\Delta t = 0.100$ s (10 Hz) nearly doubles overshoot to **18.1%** with visible ringing, and $\Delta t = 0.165$ s (6 Hz) goes **unstable** (oscillating wildly up to 1.76 m). **Rule of thumb:** sample at least 10× faster than the closed-loop rise time ($\Delta t \leq t_r/10$, i.e. $\geq$ 50–200 Hz here).

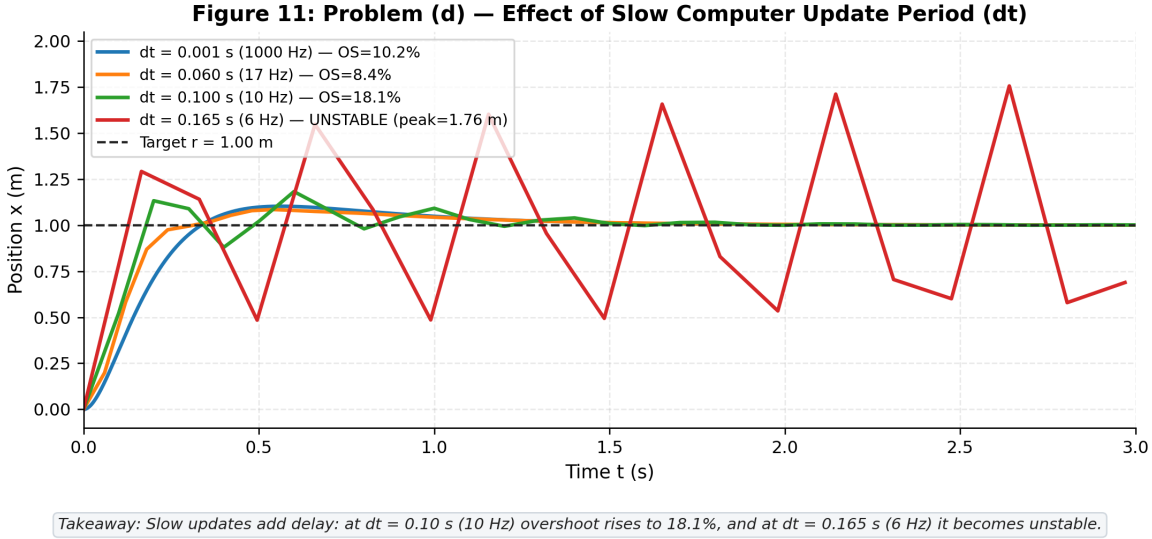


Figure 11: Increasing the sample period Δt adds phase lag, turning a clean response into unstable oscillations at 6 Hz.

8. Choosing the Gains (Manual & Ziegler–Nichols)

Step-by-step manual tuning recipe:

1. Set $K_i = 0$ and $K_d = 0$. Raise K_p until the system moves briskly toward the target with moderate bounce (around 10–25% overshoot).
2. Raise K_d (with a derivative filter τ_d if noisy) until the overshoot and bouncing disappear.

3. Raise K_i slowly until any steady-state offset vanishes within your desired settling time.
4. Fine-tune: if adding K_i brings back overshoot, nudge K_d up slightly or trim K_p by 10–20%.

Ziegler–Nichols (ZN) ultimate-gain test on a second system, $G(s) = \frac{1}{(s+1)^3}$: In time domain, this third-order plant obeys $\dddot{y} + 3\ddot{y} + 3\dot{y} + y = u$. Using P-only control, we raise K_p until the output swings steadily without growing or decaying (Figure 12, left). Analytically, the phase of $G(j\omega) = (1 + j\omega)^{-3}$ hits $-180°$ when $3\arctan(\omega_u) = 180° \Rightarrow \omega_u = \sqrt{3}$ rad/s, where $|G(j\omega_u)| = 2^{-3} = \frac{1}{8}$. Thus the exact ultimate gain and period are:

$$K_u = 8.0, \qquad T_u = \frac{2\pi}{\omega_u} = \frac{2\pi}{\sqrt{3}} = 3.628 \text{ s} \quad \text{(simulation measures } T_u = 3.630 \text{ s)}.$$

Applying the classic ZN recipe (Table 4) gives PID gains $(K_p, K_i, K_d) = (4.80, 2.646, 2.177)$. As Figure 12 (right) shows, ZN reacts fast ($t_r = 0.949$ s) because it targets "quarter-wave decay," but suffers **52.4% overshoot** and takes **9.98 s** to settle. Trimming aggression to $(K_p, K_i, K_d) = (3.40, 0.70, 2.60)$ cuts overshoot from **52.4% to 1.25%** and speeds settling from **9.98 s to 2.69 s** (3.7× faster!).

Tuning Rule	K_p	K_i	K_d	Applied Values on $1/(s+1)^3$	Result (t_r, %OS, t_s)
ZN P	$0.50\,K_u$	0	0	$(4.00, 0, 0)$	$e_{ss} = 0.20$
ZN PI	$0.45\,K_u$	$0.54\,K_u/T_u$	0	$(3.60, 1.191, 0)$	oscillatory
ZN PID	$0.60\,K_u$	$1.20\,K_u/T_u$	$0.075\,K_u T_u$	$(4.800, 2.646, 2.177)$	0.949 s, 52.41%, 9.980 s
Improved PID	Reduced K_p, K_i; higher K_d			**(3.400, 0.700, 2.600)**	**1.629 s, 1.25%, 2.690 s**

Table 4: Ziegler–Nichols recipe table and performance improvement on $G(s) = 1/(s+1)^3$.

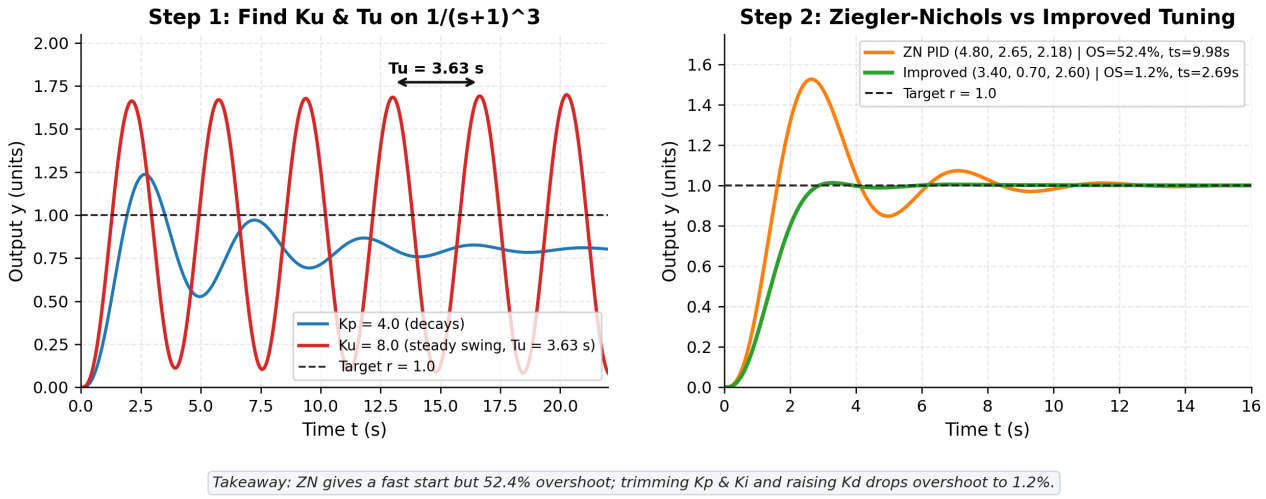


Figure 12: Left: Finding $K_u = 8.0$ and $T_u = 3.63$ s on $1/(s+1)^3$. Right: ZN PID (52.4% OS) vs. improved PID (1.25% OS).

9. The Code: Reusable PID with All Real-World Fixes

Below is the core `update()` logic from `pid_controller.py`, combining output clamping, conditional-integration anti-windup, derivative-on-measurement, and the first-order derivative filter:

```
class PIDController:
    def __init__(self, kp, ki, kd, u_min=-float("inf"), u_max=float("inf"),
                 anti_windup=True, derivative_on_measurement=True, d_filter_tau=0.0):
        self.kp, self.ki, self.kd = float(kp), float(ki), float(kd)
        self.u_min, self.u_max = float(u_min), float(u_max)
        self.anti_windup = anti_windup
        self.derivative_on_measurement = derivative_on_measurement
        self.d_filter_tau = float(d_filter_tau)
        self.reset()

    def update(self, setpoint, measurement, dt):
        error = setpoint - measurement
        p_term = self.kp * error

        # Derivative on measurement avoids setpoint kick
        if self.derivative_on_measurement:
            raw_d = 0.0 if self.prev_measurement is None else -(measurement - self.prev_measurement) / dt
        else:
            raw_d = 0.0 if self.prev_error is None else (error - self.prev_error) / dt

        # Low-pass filter on D suppresses high-frequency sensor noise
        if self.d_filter_tau > 0.0:
            alpha = dt / (self.d_filter_tau + dt)
            self.d_filtered = (1.0 - alpha) * self.d_filtered + alpha * raw_d
        else:
            self.d_filtered = raw_d
        d_term = self.kd * self.d_filtered

        # Conditional integration anti-windup
        cand_i = self.integral + error * dt
        u_cand = p_term + self.ki * cand_i + d_term
        if not (self.anti_windup and ((u_cand > self.u_max and error > 0) or
                                      (u_cand < self.u_min and error < 0))):
            self.integral = cand_i

        u = max(self.u_min, min(self.u_max, p_term + self.ki * self.integral + d_term))
        self.prev_error, self.prev_measurement = error, measurement
        return u
```

Running `python3 run_section9_demo.py` on our 1 kg test plant at 200 Hz ($\Delta t = 0.005$ s) with ± 30 N motor limit, ± 1 mm sensor noise, and a sudden step at $t = 0.3$ s produces the exact terminal output below and Figure 13:

```
=== Reusable PIDController Run (Test System) ===
Gains          : Kp=120.0, Ki=240.0, Kd=12.0
Limits / Filter: u in [-30.0, 30.0] N, tau_d=0.025 s
Rise time      : 0.483 s
Overshoot      : 0.17 %
Settling time  : 0.840 s
Steady error   : 0.0002 m
Peak force     : 30.00 N
Steady u ripple: 1.21 N (peak-to-peak)
```

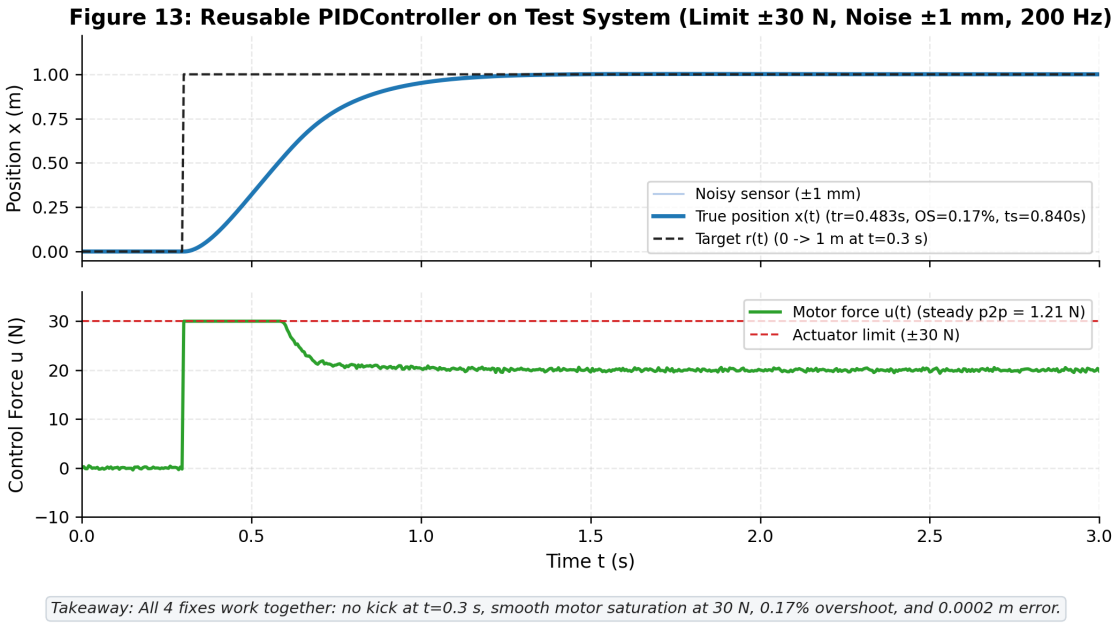


Figure 13: Production `PIDController` running with ±30 N limit, ±1 mm noise, and a step at $t = 0.3$ s.

10. Real Robots & Common Myths

Five Real-World Examples of PID Control

1. **DC Motor Speed Control:** *Setpoint:* target RPM (e.g. 1500 RPM). *Sensor:* optical encoder ticks/second. *Actuator:* PWM voltage to motor driver. *Roles:* P gives quick spin-up, I overcomes bearing friction and load torque so RPM hits target exactly, and D is often tiny or zero because speed loops are first-order and encoders are noisy.
2. **Quadcopter Drone (Attitude & Altitude):** *Setpoint:* desired roll/pitch angle (0° hover) or height (2 m). *Sensor:* IMU gyroscope/accelerometer (500–1000 Hz) and barometer/lidar. *Actuator:* four brushless motor speeds. *Roles:* P levels the drone, D damps fast angular flips from wind gusts, and I trims out battery voltage sag or off-center payload weight.
3. **Robot Arm Joint Position:** *Setpoint:* joint angle θ_{target}. *Sensor:* joint optical/magnetic encoder. *Actuator:* servo torque. *Roles:* P pulls the arm to angle, I holds the arm up against gravity (just like holding our 20 N spring!), and D prevents mechanical overshoot and ringing when stopping.
4. **Differential-Drive Line Follower:** *Setpoint:* line centered under array ($e = 0$). *Sensor:* IR reflectance sensor array. *Actuator:* left/right wheel speed difference Δv. *Roles:* P steers back toward the tape, D reacts early to sharp S-curves so the robot does not fly off, and I is usually near zero so corners do not wind up steering.
5. **Car Cruise Control:** *Setpoint:* 100 km/h. *Sensor:* wheel-speed sensor. *Actuator:* throttle opening. *Roles:* P adds gas when slowing down, I holds extra throttle climbing a long hill so speed does not sag, and D smooths transitions without jerky acceleration.

Five Common Myths About PID—Corrected

1. **Myth: "You always need all three P, I, and D."**
   Reality: Use only what the physics demands. Motor velocity and thermal loops frequently use **PI** (noise makes D counterproductive), while pure-integrator systems like some angle loops work well with **PD** (no I needed).
2. **Myth: "Higher K_p always makes a system faster and more accurate."**
   Reality: Raising K_p lowers steady-state error only up to a point; too much K_p lowers damping ratio ζ (Eq. (7)), causing violent overshoot and eventual instability.
3. **Myth: "You must know exact physics equations to use PID."**
   Reality: PID is model-free in operation—methods like manual tuning and Ziegler–Nichols (Section 8) tune a black-box machine solely by watching how it responds.
4. **Myth: "Derivative D only amplifies noise and should never be used."**
   Reality: Unfiltered D amplifies noise (39.7 N chatter in Figure 9), but adding a simple low-pass filter (τ_d) cuts chatter by $> 96\%$ while keeping D's life-saving damping.
5. **Myth: "The integral term is harmless when the actuator hits max power."**
   Reality: Without anti-windup, the integral winds up blindly during saturation (68.6 N in Figure 8), causing severe overshoot (26.9%) long after the target is reached.

Interview Preparation: Predicting What Happens When Parameters Change

In a technical evaluation or design review, you are frequently asked to predict the effect of physical parameter changes before re-running code. Table 5 provides the physical and mathematical rationale for each parameter shift:

Parameter Change	Predicted System Response	Physical & Mathematical Reason
Increase mass m	Slower rise; higher overshoot; longer settling t_s	Higher inertia; $\omega_n = \sqrt{k_{\text{tot}}/m}\downarrow$, $\zeta \propto 1/\sqrt{m}\downarrow$.
Increase spring k	Faster rise; larger steady error (if P-only)	Stiffer spring; higher natural frequency; requires larger $u_\infty = k \cdot r$.
Increase damping c	Slower rise; lower overshoot; faster settling	Adds physical damping force $-c\dot{x}$; raises damping ratio ζ.
Increase gain K_p	Faster rise; smaller e_{ss}; higher overshoot	Virtual spring stiffens; $\omega_n \uparrow$, but $\zeta \propto 1/\sqrt{k + K_p}\downarrow$.
Increase gain K_i	Faster $e_{\text{ss}} \to 0$; increased overshoot	Erases lingering gap; adds phase lag; risks windup if saturated.
Increase gain K_d	Dramatically cuts overshoot; dampens bounce	Adds virtual damping $c_{\text{eff}} = c + K_d$; raises ζ; sensitive to noise.
Lower limit $u_{\max}$	Slower initial rise; severe windup if naive	Actuator saturates longer; anti-windup clamping is mandatory.
Increase noise σ	Motor chattering and force ripple spike	D-term takes derivative of high-frequency noise; requires filter τ_d.
Increase step Δt	Increased overshoot; ringing; instability	Zero-order hold injects $\Delta t/2$ delay; erodes phase margin.

Table 5: Quick prediction matrix: how physical and controller parameters alter closed-loop performance.

Note on AI Tools: AI coding assistance was used to help draft and refine LaTeX layout, structure beginner-friendly analogies, and cross-check Python plotting code. All equations, analytical derivations (steady-state error, damping ratio ζ, ultimate gain $K_u = 8$, period $T_u = 2\pi/\sqrt{3}$), and simulation metrics were independently verified against pure-NumPy code runs.